

MemEX: An Epistemic Reasoning Graph for AI Agents

Dr. Laszlo Attila Vekony

We introduce MemEX, an open-source TypeScript library that models AI memory as a *typed, scored, provenance-tracked graph* over an append-only event log. MemEX is a practical approach to *epistemic memory* — a class of memory systems that explicitly represent what an AI system *believes*, with provenance, differential trust, contradiction, and time, rather than only what it can retrieve. Unlike vector stores, which collapse knowledge into similarity, and unlike key-value or relational stores, which collapse it into binary facts, MemEX preserves the structure that high-stakes analytical work depends on: who said it, why we believe it, what it conflicts with, when it was true, and how confident we are. We argue that this kind of structure is materially useful — sometimes critical — for AI reasoning in three particularly demanding domains: financial analysis, legal analysis, and geopolitical analysis, where the consequences of treating uncertain claims as facts, or of forgetting why a conclusion was reached, are severe. We describe a subset of MemEX's epistemic primitives (the three-axis *authority/conviction/importance* scoring, first-class edges, contradictions as data, query-time decay, and the coordinated memory/intent/task tri-graph), and we show through worked examples how these primitives can be applied to equity research with conflicting signals, common-law reasoning over superseded precedent, and scenario analysis over branching geopolitical worldlines. We close with what MemEX deliberately does not attempt and pointers to the broader design space.

A Practical Approach to Epistemic Memory in Financial, Legal, and Geopolitical Analysis

Library version: @ai2070/memex@0.11.0 · Status: Working draft for public release

> *MemEX is a practical approach to epistemic memory for AI agents, implementing a subset of properties we have found useful in agent contexts. It is one point in a broader design space — explored separately in our forthcoming concept paper, "Epistemic Memory: A Design Space for Belief-Aware AI Memory Systems."*

epistemic memory | AI agents | knowledge graph | provenance | belief representation

Introduction

The dominant pattern for "AI memory" in 2024–2026 has been retrieval-augmented generation over a vector index: embed text, retrieve the top-k by cosine similarity, paste the chunks into a prompt. This pattern works well when the question is "what does the corpus say about X?" and works poorly — sometimes catastrophically — when the question is *epistemic*: what should I **believe** about X, given that the corpus contains rumors, retractions, partisan sources, stale facts, and outright contradictions?

Three domains expose this gap unusually clearly:

- **Financial analysis** routinely combines high-authority structured data (audited filings, regulator releases) with low-authority signals (sell-side notes, anonymous Bloomberg leaks, insider chatter), and the *combination* is where alpha and risk both live. A memory that cannot tell a 10-K from a Twitter rumor cannot price either correctly.
- **Legal analysis** is fundamentally a graph: cases cite cases, statutes amend statutes, regulations interpret statutes, and *supersession* is a *first-class operation* — being overruled is not the same as being wrong. A memory that overwrites a precedent when a higher court rules has destroyed the very history the doctrine of stare decisis depends on.
- **Geopolitical analysis** routinely operates on rumored, simulated, and contested information, and the analytically interesting questions sit on long arcs — half-century rivalries, multi-decade alliance shifts — where the relevant comparison set is other historical great-power competitions. Analysts hold competing interpretive frames simultaneously ("does this rhyme with Anglo-German 1900, US-Soviet, or US-Japan 1980s?"), surface contradictions between OSINT sources and official narratives, and update confidence as events accumulate — all without flattening uncertainty into a false consensus.

In all three, the quality of an AI system is materially shaped by how faithfully its memory represents *epistemic state*, not just retrieval state.

MemEX is one practical library built with this observation in mind. It exposes a small, immutable, event-sourced API for building reasoning graphs in which every belief carries explicit provenance, every score has a meaning, every contradiction is preserved, and every change is replayable. This whitepaper situates MemEX against existing memory architectures, walks through its epistemic primitives, and demonstrates — with concrete worked examples — how those primitives can be applied in finance, law, and geopolitics. We do not claim MemEX is the only or best way to address these concerns; we describe the choices we made, why we made them, and what we deliberately did not attempt.

Who this is for

This whitepaper is written for:

- **Quantitative researchers and credit analysts** building AI tools that fuse heterogeneous evidence under audit constraints.
- **Legal-tech architects** designing systems that must respect supersession, point-in-time doctrine, and citation integrity.
- **OSINT and geopolitical analysts** who need to reason over contested information and branching scenarios without flattening uncertainty.
- **AI engineers** generally, who have outgrown vector-only memory and are looking for primitives that support belief, not just retrieval.

It assumes familiarity with TypeScript-shaped APIs but does not require expertise in any of the application domains. Code examples are illustrative; the library is small enough (~2,500 lines) to be read end-to-end.

2. Background and Related Work

2.1 Vector memory. Vector memory (e.g., FAISS, Pinecone, Weaviate, pgvector) reduces a corpus to a high-dimensional embedding space and answers "what is similar to this query?" Vector memory is *recall infrastructure*. It has no native concept of authority, contradiction, supersession, or causality. Two embeddings near each other in cosine space could be the audited filing and a satirical tweet; the index cannot tell.

2.2 Knowledge graphs. Classical knowledge graphs (RDF, Neo4j, property graphs) encode typed relationships and are a closer fit for legal and geopolitical reasoning than vectors. However, off-the-shelf graphs typically: (a) treat facts as binary, (b) store one current value per relationship, (c) provide no first-class notion of provenance or authority, and (d) rely on application code to handle conflict, decay, and identity resolution. The result, in practice, is that the *epistemic logic* gets reinvented poorly in every project.

2.3 Belief networks and probabilistic graphical models. Bayesian and Markov networks model belief explicitly with conditional probabilities. They are mathematically clean but operationally heavy: they require a fixed schema, struggle with novel entities, and conflate uncertainty about the world with uncertainty about the speaker. In financial and geopolitical settings, you often need to model both — "I am 80% sure that the analyst is 60% sure" — and to do so without a Bayesian net engineer in the loop.

2.4 Agent memory frameworks. Recent agent frameworks (LangGraph state, AutoGen memory modules, "memory" features in commercial assistants) typically expose flat key-value notes or summarized rolling buffers. They optimize for turn-to-turn continuity, not for analytical accountability. None of the commonly used frameworks, to our knowledge, treats contradiction as a first-class data type.

2.5 MemEX's positioning. MemEX is deliberately *not* a probabilistic graphical model and not a vector store. It is a graph substrate that:

1. Models knowledge as items + edges with explicit author, source kind, and three orthogonal scores.
2. Treats contradictions and supersessions as edges, not as overwrites.
3. Stores changes as an append-only command log; state is a fold over the log.
4. Computes time-decay at query time, never mutating history.
5. Coordinates three graphs — memory, intent, task — under a unified event envelope.

It is intended to sit *underneath* vector and text search, and *above* event logs and persistence layers, providing the epistemic semantics that those layers lack.

3. The MemEX Architecture

This section summarizes the primitives most relevant to the analytical applications discussed later. The library is implemented in roughly 2,500 lines of TypeScript across 18 modules; full API surface is documented in `API.md`.

3.1 The MemoryItem and its three scores. Every node in the memory graph is a `MemoryItem` (`src/types.ts:35-59`). The interface is small but carefully designed:

```
interface MemoryItem {
  id: string; // uuidv7 globally unique, time-ordered
  scope: string; // "user:laz/general", "project:xyz", etc.
  kind: MemoryKind; // observation | assertion | hypothesis
  // | derivation | policy | trait | ...
  content: Record<string, unknown>;

  author: string; // who asserted this
  source_kind: SourceKind; // user_explicit | observed
  // | derived_deterministic
  // | agent_inferred | simulated | imported
  parents?: string[]; // ids this item is derived from

  authority: number; // 0..1 system trust in the claim
  conviction?: number; // 0..1 author's stated confidence
  importance?: number; // 0..1 current operational salience

  created_at?: number;
  intent_id?: string;
  task_id?: string;
  meta?: { agent_id?: string; session_id?: string; [k: string]: unknown };
}
```

The three scores are deliberately orthogonal. **Authority** is system-level: how much should we, the system, trust the claim regardless of who said it? **Conviction** is author-internal: how sure was the source when they said it? **Importance** is operational: how much should we be thinking about this right now? In every domain we examine, all three matter and they do not co-vary.

3.2 Edges with their own authority. Edges (`src/types.ts:75-89`) are first-class objects, not properties of nodes:

```
interface Edge {
  edge_id: string;
  from: string;
  to: string;
  kind: EdgeKind; // DERIVED_FROM | CONTRADICTS | SUPPORTS
  // | ABOUT | SUPERSEDES | ALIAS
  weight?: number;
  author: string;
  source_kind: SourceKind;
  authority: number; // edges have their own trust score
  active: boolean;
  meta?: Record<string, unknown>;
}
```

The crucial point is that **edges carry their own provenance and authority**. The fact that "case A overrules case B" is itself a claim made by some author, with some confidence, citable to some source. In finance, "filing X supports thesis Y" might be a claim by a junior analyst (low authority) or by the CFO under oath (high). MemEX preserves that distinction at the edge level instead of forcing it into node metadata.

3.3 Event sourcing and immutability. MemEX is implemented as a pure reducer over commands:

```
applyCommand(state, cmd): { state: GraphState; events: MemoryLifecycleEvent[] }
```

Commands (`memory.create`, `memory.update`, `memory.retract`, `edge.*`) are stored append-only. State is reconstructed by folding commands through the reducer (`replayCommands`, `replayFromEnvelopes`). The reducer never mutates input state; every operation returns a new `GraphState` (`src/reducer.ts:69-72`). This gives the library three properties that matter enormously in regulated analytical settings:

1. **Auditability.** Every belief change can be traced to the command that caused it.

2. **Time travel.** State at any historical point is reconstructable.
3. **Branching.** Multiple worldlines can fork from the same checkpoint without contention.

3.4 Contradiction as data. Two functions (`src/integrity.ts`) frame the contradiction model:

```
markContradiction(state, idA, idB, author, meta?)
resolveContradiction(state, winnerId, loserId, author, reason?)
```

Contradictions are explicit `CONTRADICTS` edges, never overwrites. Resolution adds a `SUPERSEDES` edge from winner to loser and lowers the loser's authority — but does not delete it. Two retrieval modes (`src/retrieval.ts`) allow the application to either *filter* contradictions (deterministically keep the higher-scoring side) or *surface* them (return both, annotated with `contradicted_by`). In legal and geopolitical settings, surfacing is usually correct: the analyst needs to *see* that experts disagree, not to be presented with a confidently wrong consensus.

3.5 Query-time decay and dynamic resolution. Time decay is computed at retrieval, never written into stored scores:

```
interface DecayConfig {
  rate: number; // 0..1 per interval
  interval: "hour" | "day" | "week";
  type: "exponential" | "linear" | "step";
}
```

Because every item's id is a `uuidv7`, its creation time is recoverable without a database lookup. The retrieval pipeline (`getScoredItems`, `smartRetrieve`) accepts arbitrary `ScoreWeights` per query, so different questions can apply different decay regimes against the same graph. A "what do we know now?" query and a "what did we know in March 2024?" query are both first-class.

3.6 The memory / intent / task tri-graph. MemEX models three coordinated graphs under a unified `EventEnvelope` (`src/types.ts:108–115`). Beyond memory, there are:

- **Intents** (`src/intent.ts`): goals, with a state machine active to `paused/completed/cancelled` and `root_memory_ids` anchoring the goal in memory.
- **Tasks** (`src/task.ts`): executable units belonging to an intent, with `input_memory_ids` and `output_memory_ids` recording which beliefs were consumed and produced.

This closes a loop that flat memory systems can only fake:

```
beliefs → goals → tasks → new beliefs (with provenance back to the originating beliefs)
```

For analytical workflows, this means every conclusion carries a citable chain: "this thesis (memory) was produced by this task, which served this intent, which was opened in response to these prior observations."

3.7 Smart retrieval. `smartRetrieve` (`src/retrieval.ts`) composes the primitives above into a single ranked, budget-bounded pull:

```
smartRetrieve(state, {
  budget: number,
  costFn: (item) => number,
  weights: ScoreWeights, // includes optional decay
  filter?: MemoryFilter,
  contradictions?: "filter" | "surface",
  diversity?: { author_penalty?, parent_penalty?, source_penalty? },
})
```

The diversity penalties matter: in all three target domains, naive ranking will return five paraphrases of the same wire-service report. Penalizing duplicate authors and shared parents forces the retrieved context to span genuinely independent sources.

3.8 Transplant and multi-agent isolation. `exportSlice` and `importSlice` (`src/transplant.ts`) allow a sub-graph to be exported (with optional walking up parents and down children), handed to another agent or process, mutated, and merged back. The merge is append-only by default and supports `reIdOnDifference` to mint new `uuidv7` ids on conflict — preserving timestamp ordering by stepping forward 1 ms rather than calling `Date.now()`. This enables sandboxed sub-agent reasoning without losing main-graph integrity, which is directly relevant to red-team analysis in all three domains.

4. Concerns We Found Mattered for Analytical Work

Before turning to applications, we name four concerns that drove most of MemEX's design choices. These are not a checklist of requirements that any "epistemic memory" implementation must satisfy — the broader category is open, and other useful systems will reasonably weight things differently. Nor do we claim them to be exhaustive. They are, however, concerns we expect any practitioner working in finance, law, or geopolitics to recognize, and they are the concerns that pushed us toward the specific primitives described in §3.

1. **Differential trust.** Not all sources are equal. A useful system can rank claims by who said them, independently of how recent or how popular they are. Treating a 10-K and a Twitter rumor as semantic peers is, for most analytical purposes, a defect.
2. **Preserved disagreement.** When two credible sources contradict each other, the analyst usually needs to see both. Silent resolution — picking one and discarding the other — is a liability in any setting where the disagreement is itself a signal.
3. **Causal traceability.** Most analytical conclusions need to be answerable to: "what evidence justifies this?" — recursively, to root observations. This is sometimes a regulatory requirement, often a professional one, and almost always useful in debugging the system itself.
4. **Temporal honesty.** Claims age. Distinguishing "true now" from "true when first asserted," *without rewriting history*, matters in markets ("as of this date"), in law ("controlling authority on this date"), and in geopolitics (post-mortem on prior assessments).

A system addressing all four is not automatically sufficient for trustworthy analytical AI; calibration, detection, governance, and adversarial robustness all sit outside the substrate (see §10). What we claim is more modest: these four concerns are useful to keep in mind as we walk through the application examples, and MemEX is one practical attempt to address them at the substrate layer.

5. Application I: Financial Analysis

Financial analysis fuses heterogeneous evidence streams — corporate disclosures, conference-call commentary, third-party research, and historical pattern data — into a small set of decisions. The *quality* of the fusion determines PL. We walk through four practical examples in increasing order of how much an epistemic memory layer matters: 10-K ingestion, earnings-call interpretation, industry-report synthesis, and macro pattern analysis. The fourth — pattern-matching current conditions against historical regimes — is, in our experience, the application where the difference between an epistemic graph and a flat retrieval store is largest.

5.1 Worked example: 10-K ingestion and restatement handling. A 10-K is a mixed-authority document. Audited financial statements carry the highest authority a corporate disclosure can carry; the MDA is management's interpretation (lower authority, often forward-looking); risk factors are largely boilerplate (low authority, but occasionally a real signal); footnotes are where the interesting accounting choices live (high authority, often high importance). A naive ingestion that flattens the entire filing into similar-weight chunks loses these distinctions.

In MemEX, sections become typed observations with differentiated scores under a shared scope:

```
const auditedRevenue = createMemoryItem({
  scope: "10K:ACME-2025",
  kind: "observation",
  content: { line: "revenue", value: 9.84e9, period: "FY2025" },
  author: "filing:ACME-10K-2025",
  source_kind: "user_explicit",
  authority: 0.98,           // audited financial statement
  importance: 0.85,
});

const mdaCommentary = createMemoryItem({
  scope: "10K:ACME-2025",
  kind: "assertion",
  content: { claim: "Margin pressure from raw-material costs to ease in H2" },
  author: "filing:ACME-10K-2025",
  source_kind: "user_explicit",
});
```

```

    authority: 0.55,          // management forward-looking
    conviction: 0.8,          // management sounds confident
    importance: 0.7,
  });

const footnote12 = createMemoryItem({
  scope: "10K:ACME-2025",
  kind: "observation",
  content: { topic: "revenue_recognition_change", detail: "..." },
  author: "filing:ACME-10K-2025",
  source_kind: "user_explicit",
  authority: 0.95,
  importance: 0.9,          // accounting changes are operationally critical
});

```

Two consequences matter for analytical AI built on this ingestion:

Restatements as supersession, not deletion. When ACME later restates a prior 10-K — a frequent and material event — the new filing supersedes the old without erasing it:

```

resolveContradiction(state, auditedRevenueRestated.id, auditedRevenue.id,
  "filing:ACME-10K-2025-A", "Restated per Note 1");

```

Every derived item that consumed the original (calculated leverage ratios, covenant headroom, multi-year growth rates) appears as a stale dependent via `getStaleItems`, and `cascadeRetract` invalidates them in one call. The original observation remains queryable for "what did we believe on the original filing date?" — exactly the answer needed when reconciling historical model outputs with the new reality.

Provenance to the section, not just the document. Every later derivation — a credit rating, a thesis, a forecast — carries parents pointing to specific 10-K sections. Asked "why do we rate this BBB?" the system returns a `getSupportTree` that walks back through the calculated ratios to the audited line items and footnote disclosures that conditioned them:

```

ratingBBB
  leverageRatio (parents: [debt, ebitda])
    debt (parents: [auditedDebt]) ← audited line
    ebitda (parents: [revenue, opex]) ← audited lines
  industryOutlook (assertion, low authority)
  covenantHeadroom (parents: [debt, footnote12]) ← footnote-conditioned

```

Regulator-facing audit ("show me the chain that justified this rating on date D") becomes a state-replay question rather than a re-derivation exercise.

5.2 Worked example: earnings calls and the prepared-vs-QA authority gap. Earnings calls have an internal authority gradient that practitioners know intuitively but flat memory systems lose. Prepared remarks are scripted and lawyer-vetted (high authority, high conviction by construction). The QA is ad-libbed (lower authority, sometimes far more revealing — the moments where management hedges, deflects, or contradicts the deck are exactly the moments worth surfacing). Guidance is its own object: it supersedes prior guidance and conditions every forward model.

```

const prepared = createMemoryItem({
  scope: "call:ACME-2026Q1",
  kind: "assertion",
  content: { claim: "On track to meet full-year guidance of $10.2-10.5B" },
  author: "exec:ACME-CFO",
  source_kind: "user_explicit",
  authority: 0.85,
  conviction: 0.9,          // scripted, on-message
  importance: 0.85,
  meta: { segment: "prepared_remarks" },
});

const qa = createMemoryItem({
  scope: "call:ACME-2026Q1",
  kind: "assertion",

```

```

content: { claim: "Some softness in EMEA channel; expect Q2 recovery" },
author: "exec:ACME-CFO",
source_kind: "user_explicit",
authority: 0.65, // unscripted
conviction: 0.55, // hedged language detected
importance: 0.95, // hedging on guidance is operationally critical
meta: { segment: "qa", question_from: "analyst:morgan-stanley.k.smith" },
});

```

Two MemEX features earn their keep here.

Conviction-aware retrieval. A briefing pack querying with `weights:{authority:0.4,conviction:0.3,importance:0.3}` will surface the qa item *above* the prepared item despite lower authority, because importance and (low) conviction together capture what an analyst actually wants flagged: "the CFO sounded confident on the deck but hedged in QA." This is the cell of the score matrix that single-score systems cannot represent.

Guidance supersession, with the prior guidance still queryable. When the company issues a revised range:

```

resolveContradiction(state, newGuidance.id, oldGuidance.id,
                    "exec:ACME-CFO", "Q1 update reduced full-year range");

```

Every model that consumed the old range is automatically marked stale; the old guidance remains queryable for "what did we expect last quarter, and how have expectations evolved?" — the standard chart in any sell-side update — without bifurcating storage.

5.3 Worked example: industry reports and the citation-collapse problem. Industry reports — McKinsey, Gartner, IDC, sell-side, boutique consultancies, vendor white papers — vary widely in authority and tend to cite each other in tight loops. A 2026 report on data-center capex frequently sources its top-line number from a 2025 report that sourced it from a 2024 report whose primary source was, often, a single vendor disclosure. Vector retrieval surfaces all five as equally relevant to a query and silently collapses the citation chain.

MemEX's `parents` array preserves the chain. When report B cites report A, the ingestion pipeline records the dependency:

```

const reportA_2024 = createMemoryItem({
  scope: "industry:datacenter-capex",
  kind: "observation",
  content: { metric: "global_capex_2024", value: 2.1e11 },
  author: "report:gartner-2024-04",
  source_kind: "imported",
  authority: 0.7,
  importance: 0.6,
});

const reportB_2025 = createMemoryItem({
  scope: "industry:datacenter-capex",
  kind: "assertion",
  content: { metric: "global_capex_2024", value: 2.1e11, restated_as: "primary" },
  author: "report:mckinsey-2025-Q3",
  source_kind: "imported",
  parents: [reportA_2024.id], // explicit citation
  authority: 0.7,
  importance: 0.6,
});

```

Two consequences:

Diversity penalties stop citation collapse. A retrieval with `diversity: \ parent_penalty: 0.5, source_penalty: 0.3 \` automatically penalizes items that share a parent or source kind, so the briefing pack pulls *across* primary sources rather than returning five paraphrases of the same Gartner number.

Authority correction propagates through the chain. When the underlying primary source is later contradicted (a vendor restates capex, a methodology is questioned), `cascadeRetract` walks down the citation tree and invalidates every derived restatement. A naive index would still happily surface the four reports that cited the now-broken number.

5.4 Worked example: macro pattern analysis (regimes, analogs, and conditional positioning). We treat this example at greater length because, in our experience, *this* is the financial application where epistemic memory's advantages compound most clearly. Strategist research — the *"this rhymes with 1995" / "this looks like late-cycle 2007" / "we're in a 1970s-style stagflation regime"* style of analysis — is essentially pattern-matching current macro conditions against a library of historical episodes and reasoning conditionally on which analog fits. Practitioners do this on whiteboards and in weekly notes, but the underlying epistemic structure is an extraordinary mismatch for vector or flat-text memory.

The structure looks like this:

1. A library of historical episodes — each with rich, multi-dimensional state (yield-curve shape, Fed posture, breadth, sentiment regime, valuation, earnings-cycle position, geopolitical backdrop).
2. A current-conditions snapshot along the same dimensions.
3. A set of analog hypotheses — competing claims that current conditions resemble specific historical episodes.
4. Supporting evidence per analog — the specific dimensions where the match is strong.
5. Contradicting evidence — the specific dimensions where the match breaks down.
6. Conditional positioning — *if* analog A holds, do X; *if* analog B holds, do Y; if neither, Z.
7. Updates as new data arrives — conviction on each analog rises or falls.

This is a graph problem, not a retrieval problem. MemEX models it directly.

Historical episodes as anchored, high-authority observations. Each episode is a cluster of observations along the dimensions the desk cares about. The yield-curve shape on January 1, 1995 is a fact; what is uncertain is whether 1995 is the right analog *now*.

```
const yc1995 = createMemoryItem({
  scope: "macro:history/1995",
  kind: "observation",
  content: { feature: "yield_curve", shape: "flat", spread_2s10s: 0.05 },
  author: "data:fred",
  source_kind: "user_explicit",
  authority: 0.98,           // it happened, it's recorded
  importance: 0.3,          // baseline; rises when 1995 analog gains traction
});
// ... fed1995, breadth1995, valuation1995, sentiment1995, etc.
```

Pattern-match hypotheses with explicit support, contradiction, and conviction. The strategist's claim that current conditions rhyme with 1995 is itself a hypothesis — medium authority, with conviction reflecting how strongly the strategist sees it. A competing analog is a separate hypothesis. The two are explicitly contradicting:

```
const analog1995 = createMemoryItem({
  scope: "macro:current-regime",
  kind: "hypothesis",
  content: { claim: "Current setup rhymes with 1995 mid-cycle pause" },
  author: "strategist:internal",
  source_kind: "agent_inferred",
  parents: [
    yc1995.id, ycCurrent.id,
    fed1995.id, fedCurrent.id,
    breadth1995.id, breadthCurrent.id,
    // ... pairs of historical/current observations along each dimension
  ],
  authority: 0.5,
  conviction: 0.75,          // strategist sees it strongly
  importance: 0.95,          // drives positioning
});
```

```
const analog2007 = createMemoryItem({
  scope: "macro:current-regime",
  kind: "hypothesis",
  content: { claim: "Current setup rhymes with late-cycle 2007" },
  author: "strategist:external-bear",
  source_kind: "imported",
  parents: [/* corresponding 2007 + current pairs */],
  authority: 0.5,
  conviction: 0.7,
  importance: 0.9,
});

markContradiction(state, analog1995.id, analog2007.id, "system:regime-router",
  { rationale: "Mutually exclusive regime calls" });
```

The contradiction is preserved as data, not silently resolved. Both analogs remain live; their relative scores drive how much weight the conditional allocation gives each path.

Importance is decoupled from age — and that decoupling is the point. A naive time-decay model treats 1995 data as 30+ years stale and therefore low-relevance. That is exactly the wrong inference for macro pattern work. When the 1995 analog gains conviction, the operational *importance* of 1995 data rises, even though its creation timestamp is decades old. MemEX’s separation of importance from `created_at` supports this directly — and it supports the inverse, too: a recent observation that fits no current analog can be high authority but low importance for positioning. A periodic rebalancing of importance, conditioned on which analogs currently dominate, is one `bulkAdjustScores` call:

```
bulkAdjustScores(state,
  { scope: "macro:history/1995" },
  { importance: +0.4 }, // boost importance of 1995 episode data
  "system:regime-rebalance",
  "1995 analog conviction crossed 0.7");
```

Branching positioning via slice export. The conditional structure of strategist work — *if* 1995, do A; *if* 2007, do B — maps directly onto MemEX’s branching primitives. Each analog spawns an independent reasoning branch from the same baseline, so the desk can simulate the positioning consequences of each regime call without contaminating the consensus graph:

```
const baseline = exportSlice(memState, intentState, taskState, {
  scope_prefix: "macro:",
  include_parents: true,
});

// Branch: assume 1995 analog holds.
const { memState: branch95 } = importSlice(
  createGraphState(), createIntentState(), createTaskState(), baseline);
const positioning95 = createMemoryItem({
  scope: "macro:scenario-1995",
  kind: "derivation",
  content: { allocation: { equity: +0.05, duration: -0.10, credit: +0.02 } },
  author: "agent:portfolio-construction",
  source_kind: "derived_deterministic",
  parents: [analog1995.id /* and other 1995-conditional inputs */],
  authority: 0.8, // deterministic given the analog
  conviction: 0.75, // inherits from the analog
  importance: 0.9,
});

// Branch: assume 2007 analog holds. Same exercise, different conditioning.
```

Every conclusion in either branch carries explicit provenance back to the analog hypothesis it depends on. Asked “*why are we long duration?*” the system answers: because the 2007-analog hypothesis has conviction 0.7, the analog is supported by these specific historical/current dimension pairs, the deterministic derivation says duration outperforms in late-cycle regimes, and our

weighted positioning is therefore +X duration. That trace is generated by `getSupportTree` over the positioning item.

Conviction updates as new data arrives. Each new macro print is an observation that may strengthen or weaken existing analogs. A CPI surprise, a Fed posture change, a breadth thrust — each is a new observation; the standing intent ("update analog conviction on every new macro print") spawns a task whose output is an `applyMany` over the analog hypotheses, adjusting conviction. Old, lower-conviction analogs are not deleted; they remain queryable, and history accumulates an audit trail of *which analogs were tried and how each evolved*. Three months later, the post-mortem question — "we were wrong; which analog drove the call, and which dimensions of it broke down?" — becomes a graph traversal rather than an archeology dig through old emails and PDFs.

Why this matters in practice. A flat memory system can store the strategist's note; it cannot represent the conditional structure that makes the note actionable. A vector store can retrieve the note when queried; it cannot rank the analog hypotheses by their evidence, surface the contradicting analog, or trace a positioning conclusion back to the dimensions where the analogs agreed and disagreed. A plain knowledge graph can encode the citations; it does not natively support the conviction-and-importance decoupling that lets old data become operationally critical when present conditions resemble it. The combination of typed nodes, multi-axis scoring, contradictions-as-data, branching slices, and provenance-as-a-tree is, as far as we have seen, where macro pattern work actually lives — and the place where an epistemic substrate most visibly outperforms a retrieval-only one.

6. Application II: Legal Analysis

Common-law systems are graphs in the strict sense: cases are nodes; citations are edges; doctrine is the structure that emerges from the closure of those edges over time. Statutory and regulatory analysis is similar — statutes amend statutes, regulations interpret statutes, agency guidance interprets regulations. Contracts are mini-graphs of cross-references and defined terms, occasionally surviving across many amendments. And modern legal practice routinely operates across jurisdictions whose rules differ, contradict, or interact via treaty. We walk through four examples in increasing complexity: case law, contracts, regulatory documents, and complex multi-jurisdictional business relationships. The fourth is, in our experience, the area where flat retrieval breaks down most consequentially — and where an epistemic graph most clearly earns its keep.

6.1 Worked example: case law and precedents. A central epistemic property in legal analysis is that **being overruled is not the same as being wrong**. *Plessy v. Ferguson* (1896) was overruled by *Brown v. Board* (1954). A useful legal-analysis system needs to:

1. Mark *Plessy* as superseded for the doctrine of school segregation.
2. *Not* delete *Plessy* — it remains historically and analytically relevant.
3. Preserve the citation graph that ran through *Plessy* during 1896–1954.
4. Allow point-in-time queries: "what was the controlling authority on May 3, 1953?"

MemEX's `SUPERSEDES` edge plus its append-only log support this exactly:

```
resolveContradiction(state, brown.id, plessy.id, "court:SCOTUS",
                    "Brown v. Board of Education, 347 U.S. 483");
```

After this call: `plessy` still exists, its authority is reduced, a `SUPERSEDES` edge points from `brown` to `plessy`, and the original `CONTRADICTS` edge is retracted. A retrieval over the modern doctrinal scope filters out the superseded item; a retrieval over the 1953 worldline (using historical state from `replayCommands`) returns *Plessy* as live law.

Citation graphs as provenance. Cases cite cases. When *Brown* relies on *Sweatt v. Painter* (1950) and *McLaurin v. Oklahoma* (1950), those citations are recorded as `parents`. A brief that argues from *Brown* can be traced via `getSupportTree` through every cited authority back to the constitutional text. Asked "why does this argument rest on equal-protection grounds?" the system answers with the chain rather than a narration.

Differential authority by source kind. Within a single jurisdiction, sources have a strict hierarchy: constitutional text > statute > regulation > agency guidance > law-firm memo > blog post. This maps onto the `authority` axis combined with `source_kind`, and a retrieval pipeline for a brief can be configured:

```
smartRetrieve(state, {
  budget: 16000,
  costFn: (i) => JSON.stringify(i.content).length,
  weights: { authority: 0.85, importance: 0.15 },
  filter: {
```

```

    scope_prefix: "doctrine:1A/",
    range: { authority: { min: 0.7 } },           // exclude low-authority sources
    or: [
      { source_kind: "user_explicit" },          // primary sources
      { source_kind: "imported" },              // ingested case law
    ],
  },
  contradictions: "surface",
  diversity: { source_penalty: 0.4 },           // span across courts/circuits
});

```

The same graph powers both a brief (high-authority filter) and an issue-spotting walkthrough (low-authority included, used as a hypothesis generator) without duplicate storage.

Differential authority by jurisdiction-of-application. A 9th Circuit holding is binding within the 9th Circuit, persuasive in the 5th, and carries no formal weight outside the federal system. The same case node has different effective authority depending on the scope of the query — a property the application layer enforces by storing jurisdictional weight on edges or via per-scope filters at retrieval time. Section 6.4 develops this point in the cross-jurisdictional case.

The audit dividend. Legal advice is professionally and, increasingly, legally required to be explainable. The combination of `getSupportTree`, append-only event log, and replayable state effectively gives every conclusion a citation graph and a versioned history. When an opinion is later challenged, a firm can reconstruct the precise belief state on the date the opinion was rendered and demonstrate the chain that led to it.

6.2 Worked example: contracts (amendments, defined terms, cross-references). A contract is a small graph: clauses cross-reference clauses, defined terms are aliases for entities, schedules and exhibits are incorporated by reference, and amendments supersede or augment specific provisions while leaving the rest in force. Reviewing a contract well means traversing this graph correctly while resolving conflicts. A flat retrieval over document text loses the structure.

Conflicting clauses surfaced, not silently resolved. Two clauses can conflict — sometimes by drafter error, sometimes intentionally as carve-outs. A reviewer needs both surfaced:

```

const clauseA = createMemoryItem({
  scope: "contract:msa-2026-acme",
  kind: "assertion",
  content: { section: "8.2", text: "Liability cap: $1M aggregate" },
  author: "doc:MSA-final.docx",
  source_kind: "user_explicit",
  authority: 0.95,
  importance: 0.9,
});

const clauseB = createMemoryItem({
  scope: "contract:msa-2026-acme",
  kind: "assertion",
  content: { section: "12.4(c)", text: "Indemnity uncapped for IP infringement" },
  author: "doc:MSA-final.docx",
  source_kind: "user_explicit",
  authority: 0.95,
  importance: 0.95,
});

markContradiction(state, clauseA.id, clauseB.id, "agent:contract-reviewer",
  { rationale: "12.4(c) creates an exception to 8.2 cap" });

```

The contradiction is preserved as data. A subsequent task — e.g., a redline — can either resolve it (`SUPERSEDES`, with the lawyer as resolution author) or annotate it as an intentional carve-out via edge metadata. Crucially, the contradiction is *visible* in any future retrieval on this contract, so the next reviewer cannot accidentally read 8.2 in isolation and quote the wrong cap.

Amendments as targeted supersession. Amendment No. 3 to the MSA modifies §8.2 but leaves §12.4(c) untouched. The amendment is not a new contract; it is targeted supersession of specific provisions:

```

const clauseA_v2 = createMemoryItem({
  scope: "contract:msa-2026-acme",
  kind: "assertion",
  content: { section: "8.2", text: "Liability cap: $5M aggregate" },
  author: "doc:MSA-amendment-3.docx",
  source_kind: "user_explicit",
  parents: [clauseA.id], // explicit lineage to the original
  authority: 0.95,
  importance: 0.9,
});

resolveContradiction(state, clauseA_v2.id, clauseA.id,
  "doc:MSA-amendment-3.docx",
  "Amendment No. 3, executed 2026-Q2");

```

The original §8.2 remains queryable for "what was the cap on the date of the disputed transaction?" — a standard question in litigation. The amended version is the live clause for present advice.

Defined terms as aliases. "the Company," "ACME," "ACME Corp.," and "ACME Industries Inc." may all refer to the same entity in a contract; cross-references and audits need to know that. `markAlias` handles it:

```

markAlias(state, theCompany.id, acmeCorp.id, "agent:defined-term-resolver");
markAlias(state, acmeCorp.id, acmeIndustries.id, "agent:defined-term-resolver");
// getAliasGroup walks transitively

```

A query for "all obligations of ACME Industries Inc." resolves through the alias group and returns clauses that reference any of the equivalent forms.

Incorporated documents as parents. Schedules, exhibits, and prior agreements incorporated by reference appear as `parents` of the clauses that incorporate them. A clause that invokes "the indemnification provisions of the Master Services Agreement dated 2024-03-15" is materially dependent on a separate document; if that earlier MSA is amended or restated, `cascadeRetract` and `getStaleItems` flag the dependent clauses for re-review.

6.3 Worked example: regulatory documents. Regulatory analysis combines the supersession problem of case law with two further difficulties: (1) sources sit in a strict hierarchy of authority (constitutional text to statute to regulation to agency guidance to no-action letter to industry FAQ), and (2) multiple agencies often claim overlapping jurisdiction, with their interpretations sometimes contradicting each other and resolved only at much higher cost.

Hierarchy via authority and source kind. The same graph encodes:

```

const statute = createMemoryItem({
  scope: "regulation:US-securities",
  kind: "policy",
  content: { citation: "15 U.S.C. 78j(b)" },
  author: "congress:1934",
  source_kind: "user_explicit",
  authority: 0.99, // primary authority
  importance: 0.95,
});

const rule = createMemoryItem({
  scope: "regulation:US-securities",
  kind: "policy",
  content: { citation: "17 C.F.R. 240.10b-5" },
  author: "agency:SEC",
  source_kind: "user_explicit",
  parents: [statute.id], // promulgated under the statute
  authority: 0.92,
  importance: 0.95,
});

```

```
const noActionLetter = createMemoryItem({
  scope: "regulation:US-securities",
  kind: "assertion",
  content: { topic: "...", position: "Staff would not recommend enforcement..." },
  author: "agency:SEC-staff",
  source_kind: "user_explicit",
  parents: [rule.id],
  authority: 0.5,           // not binding precedent, but practically influential
  conviction: 0.8,         // staff stated it firmly
  importance: 0.85,
});
```

Amendments and rescissions with effective dates. A regulation passed today, effective in 12 months, is not yet binding — but it is already operative for planning. The `created_at` timestamp captures issue date; effective date lives in `meta`. A retrieval for "controlling regulation as of 2027-04-01" filters by effective date, not just creation date:

```
const ruleV2 = createMemoryItem({
  scope: "regulation:US-securities",
  kind: "policy",
  content: { citation: "17 C.F.R. 240.10b-5", version: "2026-amendment" },
  author: "agency:SEC",
  source_kind: "user_explicit",
  parents: [rule.id],
  authority: 0.92,
  importance: 0.95,
  meta: { effective_date: "2027-04-01", issue_date: "2026-04-01" },
});

resolveContradiction(state, ruleV2.id, rule.id,
  "agency:SEC", "Amended via Release 33-XXXX");
```

Both the original and amended rule remain queryable. Advice issued before the effective date refers to the original; advice for transactions after refers to the amended version. The choice is a query parameter, not a storage decision.

Multi-agency overlap as native contradiction. When the SEC and CFTC issue conflicting positions on a single instrument — the perennial crypto-classification question — the contradiction is real and operationally consequential. MemEX represents it directly:

```
markContradiction(state, secPosition.id, cftcPosition.id, "agent:regulatory-watch",
  { rationale: "Conflicting classification of token X" });
```

Both positions are surfaced in every retrieval until a court or higher authority resolves the conflict, at which point `resolveContradiction` records the resolution with the resolving authority as `author`. Until then, advice-giving systems show *both* — the right answer in a contested area, since the practical exposure depends on which regulator brings the action.

Notice-and-comment proposals as low-authority hypotheses. Proposed rules are not yet binding; they are signal-bearing about regulator intent. They map naturally onto `kind: "hypothesis"` with low authority and high importance:

```
const proposal = createMemoryItem({
  scope: "regulation:US-AI",
  kind: "hypothesis",
  content: { topic: "...", proposal: "..." },
  author: "agency:FTC",
  source_kind: "user_explicit",
  authority: 0.3,           // not binding
  conviction: 0.7,         // agency seems serious
  importance: 0.95,        // affects planning now
  meta: { stage: "notice_and_comment", comment_period_close: "2026-06-15" },
});
```

When the proposal becomes a final rule, the new item supersedes the proposal, lifting authority to ~0.9 — and any planning work

derived from the proposal is automatically flagged for re-review via `getStaleItems`.

6.4 Worked example: complex multi-jurisdictional business relationships. We treat this example at greater length because, in our experience, *this* is the legal application where epistemic memory’s advantages compound most clearly. A mid-sized multinational doing MA, restructuring, or compliance work routinely operates across five or more jurisdictions whose rules differ, whose enforcement interacts only partially, and whose treaties create selective dependencies. The legal team’s actual product — a tax structure, a data-protection compliance plan, a cross-border employment posture — is a graph over (entity × jurisdiction × rule × time) whose vertices outnumber what any flat memory can track coherently.

The structure looks like this:

1. A library of entities, each with multiple registrations and identities across jurisdictions.
2. A library of rules per jurisdiction (statutes, regulations, agency positions, case law).
3. A set of cross-jurisdictional dependencies (treaties, conflict-of-law principles, choice-of-law clauses, enforcement reciprocity).
4. A set of facts about the business — transactions, employment relationships, data flows, IP holdings — each of which has a *different legal status under each applicable jurisdiction’s rules*.
5. Conflict-of-law analyses that resolve which jurisdiction’s rules govern which fact.
6. Conditional structuring decisions: *if* we route IP through Lux, then X; *if* through DE, then Y; *if* the proposed treaty amendment passes, then Z.

This is a graph problem of the same shape as macro pattern analysis (§5.4), and MemEX’s primitives address it directly.

Scope as jurisdiction; same content, different status. The same factual claim can have different legal status under different jurisdictions, and the cleanest expression is to scope the assertion to the jurisdiction-of-application:

```
const ipTransfer_us = createMemoryItem({
  scope: "jurisdiction:US-DE",
  kind: "assertion",
  content: {
    transaction: "ip-transfer-2026-Q2",
    characterization: "ordinary income on transfer",
    rule: "Treas. Reg. 1.367(d)-1",
  },
  author: "agent:tax-counsel-US",
  source_kind: "agent_inferred",
  authority: 0.85,
  conviction: 0.85,
});

const ipTransfer_lux = createMemoryItem({
  scope: "jurisdiction:LU",
  kind: "assertion",
  content: {
    transaction: "ip-transfer-2026-Q2",
    characterization: "exempt under IP regime",
    rule: "Loi du 4 d cembre 1967 art. 50bis (legacy IP box)",
  },
  author: "agent:tax-counsel-LU",
  source_kind: "agent_inferred",
  authority: 0.85,
  conviction: 0.8,
});
```

Both assertions are valid in their respective scopes. Their cross-scope tension is itself a fact:

```
markContradiction(state, ipTransfer_us.id, ipTransfer_lux.id,
  "agent:cross-border-router",
  { rationale: "Same transaction, divergent characterization" });
```

The contradiction is what triggers the conflict-of-law analysis. It is not an error to be resolved silently; it is the entry point to the actual legal work.

Entity aliasing across registrations. ACME has a Delaware parent, a Luxembourg holding company, a UK trading subsidiary, and a Singapore branch. All are legally distinct, all are operationally one business. The legal team needs to query "all obligations of ACME" without flattening the entity hierarchy:

```
markAlias(state, acmeUS.id, acmeLux.id, "system:entity-resolver");
markAlias(state, acmeUS.id, acmeUK.id, "system:entity-resolver");
markAlias(state, acmeUS.id, acmeSG.id, "system:entity-resolver");
// getAliasGroup(state, acmeUS.id) walks transitively to all four registrations
```

A query for "all material litigation exposure of the ACME group" pulls across the alias group; a query scoped to jurisdiction:US-DE pulls only the parent's exposure. Same graph, different scoping rules.

Treaties as supersession with conditional reach. A bilateral treaty can override unilateral rules — sometimes. A US–Luxembourg double-tax treaty article that reduces domestic withholding rates is a SUPERSEDES edge whose meta records the treaty article, the protocol, and the limitation-on-benefits conditions:

```
resolveContradiction(state,
  withholding_treaty.id, withholding_us_default.id,
  "treaty:US-LU-1996-protocol-2009",
  "Article 11(2)(b) reduces withholding to 5% subject to LOB");
```

The original domestic rate remains queryable — necessary for taxpayers who fail the limitation-on-benefits test and revert to defaults. Multiple treaties can supersede the same default rule across different counterparty jurisdictions, and getSupportTree over a final rate chain shows precisely which treaty article controls.

Conditional structuring via slice export. Just as the macro example branches on competing analogs, structuring work branches on competing routings. Each candidate structure is a separate slice:

```
const baseline = exportSlice(memState, intentState, taskState, {
  scope_prefix: "deal:reorg-2026/",
  include_parents: true,
});

// Branch A: route IP through Luxembourg.
const { memState: branchLux } = importSlice(
  createGraphState(), createIntentState(), createTaskState(), baseline);
// ... add Lux-routing facts, derive tax/regulatory consequences, accumulate ...

// Branch B: route IP through Singapore.
const { memState: branchSG } = importSlice(
  createGraphState(), createIntentState(), createTaskState(), baseline);
// ... add SG-routing facts, derive tax/regulatory consequences, accumulate ...
```

Each branch is a fully independent reasoning graph rooted in a byte-identical baseline. The structuring memo presents both, with getSupportTree traces showing which rules drive each branch's outcome and where the branches diverge. When the deal closes on one branch, the unrealized branch is not deleted; it is archived for the post-mortem two years later when the team is asked "could we have routed differently and saved \$X?"

Standing intents for compliance monitoring across jurisdictions. New regulatory developments land continuously across every jurisdiction the business operates in. A standing intent — "evaluate impact of any new rule in any operating jurisdiction" — spawns tasks whose filter walks the entity-jurisdiction matrix:

```
{ scope_prefix: "jurisdiction:",
  source_kind: "user_explicit",
  range: { importance: { min: 0.7 } },
  created: { after: lastSweepTimestamp } }
```

Each task either confirms no impact or adds a `DERIVED_FROM` chain showing how the new rule modifies a downstream compliance position. The compliance team's weekly digest is generated by querying the lifecycle event stream, not by re-reading every rule.

Provenance for regulator and auditor response. When a regulator in any jurisdiction asks "explain the legal basis of this position," `getSupportTree` walks back through the local rule, the treaty (if any), the contractual choice-of-law clause, the entity registration, and the conflict-of-law analysis that produced the chosen characterization. The trace is generated, not narrated. For multi-jurisdictional matters under simultaneous review by, say, the IRS and a Luxembourg tax authority, the *same* graph supports two coherent and consistent answers — because the graph encodes which rules applied where, and the answers diverge only where the rules genuinely diverge.

Why this matters in practice. Multi-jurisdictional legal work is the area where flat retrieval fails most expensively. The same words mean different things in different scopes; the same entity has different legal identities in different registrations; the same transaction has different tax, regulatory, and contractual consequences depending on which jurisdiction's lens is applied; and the relationships between scopes (treaties, choice-of-law, enforcement reciprocity) are themselves typed graph relationships that vector retrieval cannot encode. A flat memory can index every regulation; it cannot tell you which one applies to which subsidiary's transaction at which point in time. An epistemic graph can, and does.

7. Application III: Geopolitical Analysis

Geopolitical analysis combines the trust-stratification problem of finance with the temporal-and-precedent problem of law, then adds two further demands: (1) reasoning over *contested* events where ground truth may never resolve, and (2) reasoning over *long arcs* whose only comparison set is other historical great-power competitions, none of which is a perfect analog. We walk through four examples drawn from a single half-century arc — US foreign policy toward China, 1974–2024 — in increasing scope: foundational historical context, discrete global events, multi-country regional dynamics, and long-term trend analysis. The fourth is, in our experience, the area where flat retrieval breaks down most consequentially, and where an epistemic graph most clearly earns its keep.

7.1 Worked example: historical context (foundational policy documents). US-China policy is structurally constrained by a small number of foundational documents from the 1970s and early 1980s. Every contemporary briefing, op-ed, and policy memo references them, often as `parents` of more specific positions:

- **Shanghai Communiqué (1972)** — Nixon-Zhou; the "one China" formula in which the US "acknowledges" rather than "recognizes" the PRC's position on Taiwan.
- **Joint Communiqué on the Establishment of Diplomatic Relations (1979)** — Carter; formal recognition of the PRC; ROC derecognition.
- **Taiwan Relations Act (1979)** — Congress, in parallel; statutory commitment to Taiwan's defense.
- **August 17 Communiqué (1982)** — Reagan; gradual reduction of arms sales to Taiwan, conditional on PRC peaceful approach.
- **Six Assurances (1982)** — Reagan's parallel commitments to Taiwan, communicated within weeks of the August 17 Communiqué and read for four decades since as in tension with it.

Each is a kind: "policy" item with very high authority and persistent importance:

```
const shanghaiCommunique = createMemoryItem({
  scope: "geo:US-China/foundational",
  kind: "policy",
  content: {
    citation: "Shanghai Communiqu 1972",
    operative: "US acknowledges that all Chinese on either side of the Taiwan Strait
      maintain there is but one China",
  },
  author: "doc:US-PRC-1972-02-28",
  source_kind: "user_explicit",
  authority: 0.99,
  importance: 0.95,
});
```

Two epistemic features matter immediately.

Same document, divergent interpretations. The PRC reads "acknowledge" as effectively equivalent to "accept"; the US has been careful for 50 years to maintain that "acknowledge" does not imply assent. This is a CONTRADICTS relationship between two reading-as-policy assertions:

```
markContradiction(state, prcReading.id, usReading.id, "agent:legal-analyst",
  { rationale: "Translation/interpretation gap on 'acknowledge'" });
```

The contradiction is not resolved — it is the durable structural tension that conditions every subsequent negotiation. Surfacing it in any briefing on Taiwan policy is correct.

Internal tension between parallel instruments. The August 17 Communiqué and the Six Assurances were issued within weeks of each other in 1982 and have been read as in tension ever since. They are not a strict contradiction — both are US instruments, both are "live" — but they create competing interpretive pulls. A retrieval that surfaces both, with `getSupportTree` showing each as a parent of more specific positions (arms-sale reviews, presidential transit policy, statements on Taiwan's international space), is doing the analyst's work rather than narrating around it.

Importance is decoupled from age. The 1972 Shanghai Communiqué is operationally critical to a 2024 briefing on a Taiwan crisis, despite being 52 years old. As in §5.4, importance is decoupled from creation date — query-time decay is the wrong default for foundational policy documents. A retrieval pipeline for current policy work can disable decay on `kind: "policy"` items entirely while applying it to event reporting.

7.2 Worked example: global events (Tiananmen, the spy balloon, and the early-hours problem). Discrete events drive much of geopolitical work, and each is met with multiple, mostly-conflicting characterizations from different authorities. The early hours of an event are the hardest: importance is at its peak, authority of every initial source is low or unverified, and the cost of a wrong characterization is highest.

Consider the February 2023 high-altitude balloon transit over the continental United States. In the first 48 hours the relevant memory contains:

```
const officialUS = createMemoryItem({
  scope: "geo:US-China/event-2023-balloon",
  kind: "assertion",
  content: { claim: "PRC surveillance platform; not weather-related" },
  author: "agency:US-DOD",
  source_kind: "user_explicit",
  authority: 0.85,
  conviction: 0.85,
  importance: 0.95,
});

const officialPRC = createMemoryItem({
  scope: "geo:US-China/event-2023-balloon",
  kind: "assertion",
  content: { claim: "Civilian meteorological balloon, off-course" },
  author: "agency:PRC-MFA",
  source_kind: "user_explicit",
  authority: 0.7,
  conviction: 0.8,
  importance: 0.95,
});

const osintRumor = createMemoryItem({
  scope: "geo:US-China/event-2023-balloon",
  kind: "hypothesis",
  content: { claim: "Operator is unit X based at Y" },
  author: "social:OSINT-account-23",
  source_kind: "agent_inferred",
  authority: 0.2,
  conviction: 0.6,
  importance: 0.95, // worth checking even at low authority
```

```
});

markContradiction(state, officialUS.id, officialPRC.id, "agent:event-router");
```

Three properties of MemEX make this tractable.

Dual-narrative as data, not error. Both official narratives are surfaced. A briefing query in `surface` mode returns both with `contradicted_by` populated, and an analyst building a posture brief sees the disagreement directly rather than receiving a confidently flattened "consensus."

Trust stratification across source kinds. Government statements, wire-service reports, declassified intelligence, commercial satellite imagery, on-the-ground social-media accounts, and known disinformation outlets all describe the same event. Vector retrieval treats them as semantic peers; MemEX's authority axis combined with `source_kind` encodes the analyst's trust topology directly. Disinformation sources can be assigned authority near zero *and* surfaced as `CONTRADICTS` evidence against trusted reports — itself a useful signal, since an outlet pushing back hard against a specific characterization may be evidence that the characterization is correct.

Importance-driven attention without authority inflation. The OSINT rumor about specific operator attribution is low authority, high importance — *worth checking*, not *worth trusting*. The analyst's standing intent targets exactly this cell of the score matrix:

```
{ scope_prefix: "geo:US-China/",
  range: { authority: { max: 0.3 }, importance: { min: 0.85 } } }
```

Items matching the filter trigger verification tasks whose results either raise authority on the rumor or refute it — and either way, the chain is auditable.

Cluster suppression via diversity penalties. Within hours, dozens of outlets paraphrase a single primary report. `parent_penalty` and `source_penalty` ensure the briefing pack pulls *across* primary sources rather than returning twenty paraphrases of the same Reuters wire.

The same primitives apply to events still being re-litigated decades on. The June 1989 Tiananmen events have a body of competing narratives — official PRC silence and management, contemporaneous Western press accounts, declassified diplomatic cables released years later, dissident memoirs, retrospective state-media reframing — each with different authority and provenance, all queryable by event scope. The graph supports both "what was the contemporaneous Western press account?" and "what do we now know post-declassification?" as point-in-time queries against the same store.

7.3 Worked example: multi-country regional dynamics. US-China policy is never bilateral. The two governments are nodes in a dense network of allied, partnered, hedging, and adversarial relationships, and a posture decision in Washington toward Beijing depends on, and conditions, postures across at least a dozen other dyads: US-Japan, US-ROK, US-Taiwan, US-Australia (AUKUS), US-Philippines (EDCA), US-India (Quad), US-EU, China-Russia, China-DPRK, China-Pakistan, China-ASEAN, China-Iran, and others. A flat memory keyed on "China policy" loses this network entirely.

****Scope per dyad; parents for cross-dyad dependencies.**** Each bilateral has its own scope:

```
const usJapanDefense = createMemoryItem({
  scope: "geo:US-Japan/defense-posture",
  kind: "assertion",
  content: { claim: "Japan defense spending to reach 2% of GDP by FY2027" },
  author: "agency:JPN-MOFA",
  source_kind: "user_explicit",
  parents: [
    chinaMilitaryBuildup.id,           // scope: geo:China/military
    dprkMissileProgram.id,             // scope: geo:DPRK/nuclear
    usJapanTreaty1960.id,              // scope: geo:US-Japan/foundational
  ],
  authority: 0.9,
  importance: 0.9,
});
```

Asked "why is Japanese defense spending rising?" `getSupportTree` returns the chain across three other scopes — China buildup, DPRK trajectory, treaty obligations — rather than a narration. The same primitive that supports legal multi-jurisdictional reasoning (§6.4) supports geopolitical multi-actor reasoning here.

Coalitions as alias groups. Multilateral arrangements like the Quad (US, Japan, India, Australia), AUKUS (US, UK, Australia), or Five Eyes (US, UK, Canada, Australia, NZ) are alias groups across bilateral scopes:

```
markAlias(state, quadUS.id, quadJapan.id, "system:coalition-resolver");
markAlias(state, quadUS.id, quadIndia.id, "system:coalition-resolver");
markAlias(state, quadUS.id, quadAustralia.id, "system:coalition-resolver");
// getAliasGroup(quadUS.id) returns the four-member group
```

A query for "Quad joint statements on Taiwan" walks the alias group; a query scoped to `geo:US-Japan` returns only US-Japan-specific items.

Cross-dyad contradiction as input to coherence checks. Coalition partners often disagree, sometimes loudly. EU positions on China economic policy diverged from US positions through the late 2010s and early 2020s, even as security cooperation deepened. The contradictions are real and policy-relevant:

```
markContradiction(state, usEconomicPosition.id, euEconomicPosition.id,
    "agent:coalition-watch",
    { rationale: "Decoupling vs de-risking framing" });
```

Surfacing the disagreement in any briefing on transatlantic China policy is the analyst's job; silently flattening it produces wrong advice.

Standing intents for regional sweep. A weekly "regional posture review" intent spawns tasks across every dyad in the watched matrix and produces a digest from the lifecycle event stream — not by re-reading every cable.

7.4 Worked example: long-term trends (the engagement-to-competition arc and historical analogs). We treat this example at greater length because, in our experience, *this* is the geopolitical application where epistemic memory's advantages compound most clearly. The half-century arc of US policy toward China — engagement beginning under Nixon, deepening through normalization, WTO accession, and economic interdependence, then shifting through the Pivot to Asia, the trade war, technology decoupling, and full strategic competition — is exactly the kind of long-arc analysis that is built on (a) competing interpretive narratives about what the trajectory has been, (b) competing analogs to historical great-power competitions, and (c) conditional forward scenarios depending on which interpretation one accepts. None of this is well-served by retrieval over text.

The structure looks like this:

1. A library of dimensional time-series across the full arc — bilateral trade, mutual investment, military spending, diplomatic exchange volume, technology trade restrictions, alliance architecture, ideological framing.
2. A library of competing macro narratives — "engagement worked / engagement failed," "China is rising / China is peaking," "decoupling / de-risking / re-coupling," "Thucydides Trap / no trap."
3. A library of historical great-power competition analogs — late-19th-century Anglo-German naval and industrial rivalry, US-Soviet Cold War, US-Japan 1980s economic competition, Athens-Sparta — each with rich dimensional state.
4. Pattern-match hypotheses asserting that the current US-China dynamic resembles one or more of these analogs, with explicit support and contradicting evidence per dimension.
5. Conditional forward scenarios — *if* the Anglo-German analog dominates, expect X over Y horizon; *if* US-Soviet, then Z; *if* US-Japan, then W.
6. Updates as new events arrive — each event strengthens or weakens specific analogs.

This is structurally identical to macro pattern analysis in finance (§5.4), with the dimensions changed and the time horizon stretched.

Historical analogs as anchored, high-authority observations. Each prior great-power competition is a cluster of observations along the dimensions strategists care about:

```
const angloGerman_naval = createMemoryItem({
  scope: "geo:history/anglo-german-1890-1914",
  kind: "observation",
  content: { dimension: "naval_arms_race", feature: "tonnage_growth_rate", value:
    0.07 },
  author: "data:historical-corpus",
  source_kind: "user_explicit",
  authority: 0.95,
```

```

    importance: 0.4,          // baseline; rises if Anglo-German analog dominates
  });
// ... us-soviet ideological competition, us-japan 1980s economic rivalry, etc.

```

Pattern-match hypotheses with explicit support and contradiction. Each analog claim is a hypothesis with parents linking historical and current observations along matched dimensions:

```

const angloGermanAnalog = createMemoryItem({
  scope: "geo:US-China/macro-frame",
  kind: "hypothesis",
  content: { claim: "Current US-China dynamic rhymes with Anglo-German rivalry
    1890-1914" },
  author: "strategist:internal",
  source_kind: "agent_inferred",
  parents: [
    angloGerman_naval.id, currentNavalBalance.id,
    angloGerman_economic.id, currentEconomicInterdep.id,
    angloGerman_alliance.id, currentAllianceArch.id,
  ],
  authority: 0.5,
  conviction: 0.6,
  importance: 0.9,
});

const usSovietAnalog = createMemoryItem({
  scope: "geo:US-China/macro-frame",
  kind: "hypothesis",
  content: { claim: "Current US-China dynamic rhymes with US-Soviet competition" },
  author: "strategist:external-hawk",
  source_kind: "imported",
  parents: [/* matched US-Soviet + current pairs */],
  authority: 0.5,
  conviction: 0.7,
  importance: 0.9,
});

const usJapanAnalog = createMemoryItem({
  scope: "geo:US-China/macro-frame",
  kind: "hypothesis",
  content: { claim: "Manageable through institutional channels; analog is US-Japan
    1980s" },
  author: "strategist:engagement-school",
  source_kind: "imported",
  parents: [/* matched US-Japan + current pairs */],
  authority: 0.5,
  conviction: 0.5,
  importance: 0.85,
});

markContradiction(state, angloGermanAnalog.id, usJapanAnalog.id,
  "agent:macro-frame-router",
  { rationale: "Hostile-vs-manageable competition framing" });
markContradiction(state, usSovietAnalog.id, usJapanAnalog.id,
  "agent:macro-frame-router",
  { rationale: "Ideological-vs-economic framing" });

```

All three analogs remain live. The surface retrieval mode returns them with `contradicted_by` populated, and a posture brief is written *to* the disagreement rather than around it.

Interpretive narratives as separate hypotheses. Independent of analog choice, narratives like "engagement worked" / "engagement failed" / "engagement was a phase" are separate hypotheses that can SUPPORT or CONTRADICT each analog claim:

```
const engagementFailed = createMemoryItem({
  scope: "geo:US-China/macro-narrative",
  kind: "hypothesis",
  content: { claim: "1972-2018 engagement did not produce political liberalization
    or strategic convergence" },
  author: "strategist:external-hawk",
  source_kind: "imported",
  parents: [
    tiananmen1989.id, hongkongNSL2020.id,
    militaryBuildupTrajectory.id, scsBuildup2014.id,
  ],
  authority: 0.55,
  conviction: 0.8,
  importance: 0.9,
});
```

A SUPPORTS edge from this narrative to the US-Soviet analog and a CONTRADICTS edge against the US-Japan analog make the structure explicit rather than implicit in prose.

Importance updates as analogs gain conviction. When a current event substantially strengthens one analog over the others — a Taiwan crisis with naval escalation strengthens Anglo-German; a sustained ideological-export effort strengthens US-Soviet; a successful institutional negotiation strengthens US-Japan — the operational importance of the corresponding historical data rises, *exactly mirroring the macro-pattern logic of §5.4*:

```
bulkAdjustScores(state,
  { scope: "geo:history/anglo-german-1890-1914" },
  { importance: +0.4 },
  "system:analog-rebalance",
  "Anglo-German analog conviction crossed 0.7 after 2027 Taiwan exercise");
```

Conditional forward scenarios via slice export. Forward planning under analog uncertainty branches on the dominant frame:

```
const baseline = exportSlice(memState, intentState, taskState, {
  scope_prefix: "geo:US-China/",
  include_parents: true,
});

// Branch A: Anglo-German analog dominates      naval-arms-race world.
// Branch B: US-Soviet analog dominates         bipolar-bloc world.
// Branch C: US-Japan analog dominates          managed-interdependence world.
```

Each branch is an independent reasoning graph; each generates a posture recommendation traceable via `getSupportTree` to the analog hypothesis it depends on. Asked "why are we recommending tighter export controls on advanced semiconductors?" the system answers: because the US-Soviet analog has crossed conviction 0.65, the analog is supported by these specific dimension matches, technology decoupling is the historical signature of bipolar-bloc competition, and our recommendation therefore extends prior controls. The trace is generated, not narrated.

Conviction updates as new events arrive. Each significant event — Pelosi Taiwan visit (2022), AUKUS announcement (2021), Phase One trade deal (2020), spy balloon (2023), Xi-Biden Woodside meeting (2023) — is an observation that updates analog conviction. The standing intent ("update all macro-frame hypotheses on each major event") spawns a task that runs `applyMany` over the analog hypotheses, adjusting conviction. Old, lower-conviction analogs are not deleted; they remain queryable, and history accumulates an audit trail of *which analogs were active when, and how each evolved across the arc*. A 2024 retrospective question — "in 2018, which analog dominated, and which dimensions of it have since broken down?" — is a graph traversal, not an archeology dig through old memos.

Why this matters in practice. A flat memory system can store 50 years of cables and reports; it cannot represent the conditional structure that makes any of it actionable for current decisions. A vector store can retrieve documents adjacent to a query; it cannot rank competing analog hypotheses by their evidence, surface the contradicting analog, or trace a posture recommendation back to

the dimensions where the analogs agreed and disagreed. A plain knowledge graph can encode the citations across the historical record; it does not natively support the conviction-and-importance decoupling that lets 1900s naval data become operationally critical when present-day naval dynamics resemble it. The combination of typed nodes, multi-axis scoring, contradictions-as-data, branching slices, and provenance-as-a-tree is, as far as we have seen, where long-arc strategic analysis actually lives — and where the half-century US-China arc is a representative, rather than exotic, case.

8. Agent Patterns: Crews, Swarms, Cross-Session Memory, and Background Thinking

The combination of an append-only event log, in-whole-or-in-part exportable graph state, append-only import with conflict detection and re-id, and the coordinated memory/intent/task tri-graph supports several multi-agent patterns directly — without requiring application code to reinvent the synchronization, provenance, or conflict-resolution logic. We outline four such patterns. Each is something practitioners build today on top of memory systems that lack the primitives; in MemEX they fall out of the design.

8.1 Crews — members operating on subsets of the graph. A crew is a small group of agents (or human-agent pairs) that share a long-running goal but specialize on different parts of the graph. A typical legal example: a US tax-counsel agent works `scope_prefix: "jurisdiction:US-DE"`, a Lux specialist works `scope_prefix: "jurisdiction:LU"`, a transfer-pricing specialist works `scope_prefix: "deal:reorg-2026/transfer-pricing"`, and a partner agent reviews everything. None of them needs the full graph in working context; each pulls a focused slice via filters or `exportSlice`:

```
const usSlice = exportSlice(memState, intentState, taskState, {
  scope_prefix: "jurisdiction:US-DE",
  include_parents: true,           // walk up to dependencies
});
// Hand to tax-counsel-US agent; the agent operates on its own GraphState copy.
// On return, importSlice merges back with conflict detection.
```

Because import is append-only by default and supports `reIdOnDifference`, a crew member working on a stale slice cannot accidentally clobber the consensus graph — divergent edits are minted as new `uuidv7` ids and surface as conflicts in the import report. Coordination becomes data, not chatter: the partner agent can query *which* member produced *which* item by `meta.agent_id` and `author`, and reconciliation is a graph operation rather than a meeting.

8.2 Swarms and subagents — parallel exploration. Where a crew is a small, durable team, a swarm is a transient, fan-out pattern: a parent agent spawns *N* subagents on the same baseline slice, each runs a different reasoning path, and the parent merges results with conflict detection. The branching pattern from §6.4 (route IP through Lux vs Singapore) and §7.4 (Anglo-German vs US-Soviet vs US-Japan analog) is exactly this primitive applied at coarse granularity; the same primitive scales down to fine-grained "explore three drafts of this clause in parallel" or "run these five hypothesis-verification subqueries simultaneously."

```
// Spawn three subagents on the same baseline.
const baseline = exportSlice(memState, intentState, taskState, {
  scope_prefix: "deal:reorg-2026/",
  include_parents: true,
});

const subagentResults = await Promise.all(
  hypotheses.map(h => runSubagent({ slice: baseline, hypothesis: h }))
);

// Merge back. Each subagent's slice has new derivations rooted in baseline.
let s = { mem: memState, intent: intentState, task: taskState };
for (const out of subagentResults) {
  const merged = importSlice(s.mem, s.intent, s.task, out.slice,
    { reIdOnDifference: true });
  s = { mem: merged.memState, intent: merged.intentState, task: merged.taskState };
  // merged.report distinguishes created/updated/conflicts per entity
}
```

Subagent isolation is structural, not a convention. Subagents operate on copies of the graph, so their reasoning cannot contaminate each other's working state. Merging is the explicit, auditable step where convergence happens — and the import report is the system's record of *which subagent contributed what, and where their conclusions disagreed*.

8.3 Cross-session and cross-chat memory. A user's relationship with an AI system spans many conversations and many days. Vector memories typically address this by storing summaries; flat key-value memories address it by accumulating notes. Both lose the structure that matters across sessions: which beliefs were established when, by whom, with what conviction, and on what evidence — and which beliefs from prior sessions were superseded or retracted later.

MemEX treats sessions as a meta field on lifecycle events and as a first-class scope dimension when the application wants it. Beliefs persist across sessions by default; nothing is "in conversation memory" vs "in long-term memory" — there is one graph, and its content is filtered or scored at retrieval time.

A new session begins by querying the graph the user has built up across all prior sessions:

```
smartRetrieve(state, {
  budget: 4000,
  costFn: (i) => JSON.stringify(i.content).length,
  weights: {
    authority: 0.4,
    importance: 0.4,
    decay: { rate: 0.05, interval: "day", type: "exponential" },
  },
  filter: { scope_prefix: "user:laz/" },
  contradictions: "surface",
});
```

This pulls the user's most operationally-important, still-fresh, non-superseded beliefs into the new conversation's working context. Items the user has *retracted* in earlier sessions (`memory.retract` commands stored in the event log) do not appear, because retraction is structural rather than a tag. Items that were merely *amended* are surfaced in their current form, but the prior version remains queryable for "what did we think before?"

The practical consequence: continuity is the default and forgetting is the operation that has to be invoked explicitly. A user who in week 1 told the agent "I prefer Python over TypeScript" need not repeat themselves in week 6 — and if their preference shifts, an update or `retract` command captures *that* fact too, with its own provenance.

8.4 Background thinking operations. The pattern that ties the others together — and the one that most distinguishes a memory built around belief from a memory built around retrieval — is *autonomous reasoning that the user did not ask for*. A traditional assistant only thinks when prompted. An agent built on an epistemic graph can think between prompts: identify high-importance, low-resolution items in its memory and work to resolve them, surface contradictions that have not yet been examined, sweep for stale derivations whose roots have been retracted, run scenario branches against new events, and pre-compute the briefing the user is likely to ask for next.

The mechanism uses primitives that already exist in the library:

- **A soft token/credit budget** per hour or per day — a cap on how much background work the agent does, refilling on a leaky-bucket schedule.
- **Prioritization driven by the score axes** — items are picked for thinking by approximately $\text{importance} \times (1 - \text{authority})$, so the agent works on what it does not yet trust well enough but cares about most. Conviction-uncertainty (an author claiming high conviction on a belief the system has low authority on) is a particularly strong attractor.
- **Alignment with intents** — every background task is created against an existing `Intent` (or, where a clear gap is identified, opens a new sub-intent traceable to a user goal). Background thinking is goal-anchored, not free-running.
- **Scheduled execution** — tasks pull from the queue, run in the background, and on completion produce new memory items with `parents` linking to the items they consumed and `intent_id / task_id` linking to the goal they served. The full chain is auditable.
- **Notification on completion or surprise** — the agent surfaces results to the user when (a) a tracked intent has new high-authority output, (b) a scenario branch has resolved with a clearly preferred outcome, (c) a contradiction it has been trying to verify has been confirmed or refuted, or (d) an unexpected high-importance item has been derived that the user did not ask for but that crosses an attention threshold.

In MemEX terms, the scheduler loop looks roughly like:

```
// Pull candidates for background attention: low-trust, high-importance items.
const candidates =.getItems(memState, {
  range: { authority: { max: 0.5 }, importance: { min: 0.7 } },
});

// Prioritize by importance (1 - authority); skip items that already
// have an open task; respect the credit budget for this window.
const pending = candidates
  .map(item => ({
    item,
    priority: (item.importance ?? 0) * (1 - (item.authority ?? 0)),
  }))
  .filter(c => c.priority > attentionThreshold)
  .filter(c => !hasOpenTaskFor(taskState, c.item.id))
  .sort((a, b) => b.priority - a.priority)
  .slice(0, withinBudget(creditBudget));

// For each, open a task under the relevant intent.
for (const c of pending) {
  applyTaskCommand(taskState, {
    type: "task.create",
    task: {
      id: uuidv7(),
      intent_id: relevantIntentFor(c.item),
      action: "verify_or_resolve",
      status: "pending",
      input_memory_ids: [c.item.id],
      priority: c.priority,
    },
  });
}
```

When a task completes, the produced memory items carry `parents = [c.item.id, ...evidence]` and `intent_id` and `task_id`, so any user query about the new derivation walks back to the original triggering observation, the intent it served, and every piece of evidence consumed along the way. The user sees only what crosses the notification threshold; the system has a full audit trail of *what it thought about, when, and why* — even for the work that produced no surfaced result.

This pattern — bounded autonomous reasoning, prioritized by belief uncertainty, anchored to user intent, surfaced by interest — is what an "agent that thinks for you" actually means in practice. It is not a feature MemEX provides. It is a pattern MemEX makes constructible from primitives that the application layer composes. We call it out because, in our view, the value of an epistemic memory layer compounds most clearly when the agent built on it is allowed to reason between user prompts. A retrieval-only memory cannot tell the agent what is worth thinking about. A belief-aware memory can, and does.

9. Cross-Domain Synthesis

The three applications above are not three separate use cases; they exercise the *same* primitives.

The point is not that MemEX is uniquely capable in any single dimension — there exist law-firm tools that handle supersession, quant systems that handle point-in-time data, OSINT platforms that handle source weighting. The point is that the *same small library* exposes all of these as composable primitives. An AI agent reasoning across the three domains — for example, a sovereign-debt analyst combining geopolitics, regulatory law, and credit — can use a single substrate.

10. Limitations and Open Problems

We list candidly the things MemEX does not solve and the things that remain open research.

9.1 Authority calibration. The library does not assign authority for you. A naive ingestion pipeline that imports every source at `authority: 0.7` defeats the purpose. Authority must be calibrated per source kind, ideally per source, ideally tracked over time as sources are validated or invalidated. Building this calibration layer is ongoing work; existing implementations rely on hand-curated source registries and post-hoc Bayesian updating against ground truth events.

Need	Finance	Law	Geopolitics	MemEX primitive
Differential trust	10-K vs. tweet	SCOTUS vs. blog	NYT vs. troll	authority
Author confidence vs. system trust	analyst conviction	dictum vs. holding	analyst caveats	conviction independent of authority
Salience without endorsement	rumor worth checking	unsettled doctrine	unverified field report	importance independent of authority
Provenance	audit trail to filings	brief citations	OSINT chains	parents + getSupportTree
Disagreement preserved	bull/bear theses	conflicting clauses	contradicting OSINT	CONTRADICTS edge + surface mode
Supersession without deletion	restatements	overruled cases	retracted reports	SUPERSEDES edge
Temporal honesty	point-in-time	"as of" doctrine	scenario timing	query-time DecayConfig
Branching	shadow portfolios	alternative arguments	scenario worldlines	exportSlice/importSlice
Goal-tracked work	thesis verification	brief drafting	verification ops	Intent + Task graphs
Avoiding source collapse	fund-of-fund correlations	over-citing one circuit	wire-service echo	diversity penalties

9.2 Contradiction detection. The library makes contradictions easy to *represent* but does not detect them. Detection is a hard, domain-specific NLP problem. In legal text, contradiction detection between clauses is a well-studied subfield; in OSINT, it is closer to an open research area. MemEX is a substrate that consumes detection results; it is not a detector.

9.3 Decay choice. The three decay models (exponential, linear, step) are coarse. Real-world claim half-lives vary enormously by domain — a tactical OSINT report decays in hours, a constitutional precedent in centuries — and per-class decay configuration is the responsibility of the application. We have not yet seen a principled framework for learning decay parameters from outcome data.

9.4 Scale. The current implementation holds state in in-memory Maps. The serialization layer (toJSON/fromJSON) and event-replay primitives mean that persistence is straightforward but external. Distributed reasoning across very large graphs would require a partitioning story that the library does not currently provide.

9.5 Probabilistic semantics. MemEX is not a probabilistic graphical model. The three scores are not probabilities and combining them via ScoreWeights is a heuristic, not a principled posterior update. For applications where the math matters (e.g., portfolio construction with explicit subjective probabilities), MemEX should sit beside a Bayesian inference engine, not replace it.

9.6 Adversarial inputs. In all three domains, adversaries actively shape the source landscape. An advanced state actor can flood OSINT, a sophisticated counterparty can flood the analyst's inbox, a hostile litigant can paper the record. The library has no built-in defense against coordinated low-authority spam — but its bulkAdjustScores and applyMany primitives make it tractable to write *retrospective* mitigations as rule-based policies once attack patterns are identified.

Getting Started

MemEX ships as @ai2070/memex on npm:

```
npm install @ai2070/memex
```

The library has a single runtime dependency (uuidv7) and an optional peer dependency on zod for validation. The repository contains:

- README.md — quick-start and canonical end-to-end workflow.
- API.md — full public API reference.
- MEMEX_DESIGN.md — design rationale and the framing this whitepaper builds on.
- MEMEX_SPEC.md — formal command/event protocol.
- memory-events-spec.md — event envelope details.

The codebase is intentionally small and is intended to be read end-to-end by engineers evaluating it for adoption.

A minimal end-to-end usage example, showing creation, derivation, scored retrieval, and slice export, is at the end of README.md and in tests/.

Further Reading

- MemEX source repository — @ai2070/memex@0.11.0 on npm.
- *uuidv7* IETF draft — time-ordered universally unique identifiers, used for ids and query-time decay.
- Pearl, J. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann, 1988. Background for the probabilistic-graphical-model alternative discussed in §2.3.
- Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. The RAG baseline contrasted in §1.
- *Forthcoming*: "Epistemic Memory: A Design Space for Belief-Aware AI Memory Systems." A concept paper situating MemEX as one practical approach within a broader design space.

Disclosure and Status

MemEX is developed by the author of this whitepaper. This document articulates the design rationale and intended use of the library; it is not a peer-reviewed evaluation, and it deliberately does not include comparative benchmarks. Empirical evaluation against alternative memory architectures is open work, and we welcome external comparisons.

Worked examples in §§5–7 are illustrative scenarios constructed for pedagogical purposes. Nothing in this whitepaper constitutes financial, legal, or political advice in any of the domains discussed.

Conclusions

The dominant patterns for AI memory have been optimized for *retrieval*, not for *belief*. In domains where the difference between a fact and a rumor is measured in millions of dollars (finance), in client liberty (law), or in lives (geopolitics), retrieval optimization on its own is insufficient. Analytical AI in these domains benefits substantially from a memory substrate that preserves provenance, authority, conviction, salience, contradiction, supersession, and time, and that does so in a way that is auditable and replayable.

MemEX is one practical approach. By committing to a small set of primitives — three orthogonal scores, first-class edges with their own provenance, append-only event-sourced state, query-time decay, contradictions and supersessions as data, a coordinated tri-graph for memory/intent/task, and exportable slices for sandboxed sub-agent reasoning — it provides a substrate that practitioners can use today across all three domains we have examined. We do not claim it is the only way, or the right way for every use case. We claim it is a useful way, that it ships, and that it is small enough to read, fork, and replace where its choices do not fit.

The library is ~2,500 lines, pure (no I/O, no networking), and deliberately unopinionated about persistence and search. It implements a subset of what we believe an epistemic memory layer for AI agents can usefully provide; the broader design space — including alternatives we have not built and trade-offs we have not made — is the subject of a separate concept paper, *Epistemic Memory: A Design Space for Belief-Aware AI Memory Systems*. We invite practitioners and researchers to evaluate MemEX against their own use cases, and we welcome implementations that explore other corners of the space.

ABOUT THIS MANUSCRIPT

This manuscript was prepared using [Rxiv-Maker v1.22.0 \(1\)](#).

EXTENDED AUTHOR INFORMATION

Bibliography

1. Bruno M. Saraiva, António D. Brito, Guillaume Jaquemet, and Ricardo Henriques. Rxiv-maker: an automated template engine for streamlined scientific publications, 2025. doi:[10.48550/arXiv.2508.00836](#).